

JavaScript Scope, Hoisting, TDZ & Strict Mode

Section 11 — Scope & Hoisting

এই chapter-এর মূল লক্ষ্য হলো JavaScript কীভাবে **variables** কোথায় **accessible** হবে, কখন **available** হবে, এবং **declaration**-এর আগে **access** করলে কী হয়—এগুলো পরিষ্কারভাবে বোঝা।

1. Scope কী?

Scope হলো একটি variable/function কোথায় থেকে access করা যাবে তার boundary বা সীমা।

সহজভাবে:

Scope = কোন জায়গা থেকে কোন **variable**-কে **access** করতে পারব।

Example:

```
const name = "Shariar";

function greet() {
  console.log(name);
}

greet();
```

এখানে `name` global scope-এ আছে।

তাই `greet()` function-এর ভিতর থেকেও `name` access করা যাচ্ছে।

2. JavaScript-এর প্রধান Scope

JavaScript-এ গুরুত্বপূর্ণ scope:

1. Global Scope
 2. Function Scope
 3. Block Scope
 4. Lexical Scope
 5. Scope Chain
-

3. Global Scope

যে variable সবচেয়ে বাইরের level-এ declare করা হয়, সেটি সাধারণভাবে **global scope**-এ থাকে।

```
const username = "Shariar";

function greet() {
  console.log(username);
}

greet();
```

Output:

Shariar

কারণ `username` global scope থেকে function-এর ভিতরে accessible।

Global variable

```
const appName = "My App";

console.log(appName);
```

এখানে `appName` global scope-এ।

Function-এর ভিতরেও:

```
const appName = "My App";

function showApp() {
  console.log(appName);
}

showApp();
```

Output:

4. Global Scope-এর সমস্যা

অনেক বেশি global variable ব্যবহার করা ভালো practice নয়।

```
let username = "Shariar";  
let age = 31;  
let score = 100;  
let status = "active";
```

বড় application-এ অনেক global variable থাকলে:

- নাম collision হতে পারে
- কোন code variable পরিবর্তন করেছে বোঝা কঠিন হতে পারে
- debugging কঠিন হয়
- unexpected behavior হতে পারে

তাই প্রয়োজন অনুযায়ী scope ছোট রাখা ভালো।

5. Function Scope

Function-এর ভিতরে declare করা variable সাধারণত সেই function-এর ভিতরেই accessible।

```
function greet() {  
  const message = "Hello";  
  
  console.log(message);  
}  
  
greet();
```

Output:

Hello

কিন্তু:

```
function greet() {  
  const message = "Hello";  
}  
  
console.log(message);
```

এখানে error হবে:

ReferenceError

কারণ `message` function scope-এর ভিতরে।

6. Function Scope এবং `var`

`var` হলো **function-scoped**।

```
function test() {  
  var x = 10;  
  
  if (true) {  
    var y = 20;  
  }  
  
  console.log(x);  
  console.log(y);  
}  
  
test();
```

Output:

```
10  
20
```

খেয়াল করো:

`if` block-এর ভিতরে `y` declare হলেও `var` block scope মানে না।

কারণ `var` function scope follow করে।

7. Block Scope

`{ }` দিয়ে তৈরি block-এর ভিতরের scope-কে block scope বলা হয়।

Example:

```
if (true) {  
  let message = "Hello";  
  
  console.log(message);  
}
```

এখানে `message` block-এর ভিতরে accessible।

কিন্তু:

```
if (true) {  
  let message = "Hello";  
}  
  
console.log(message);
```

এখানে:

ReferenceError

কারণ `let` block-scoped।

8. `let` এবং `const` Block Scoped

```
{  
  let a = 10;  
  const b = 20;  
  
  console.log(a);  
  console.log(b);  
}
```

Block-এর ভিতরে দুটোই accessible।

কিন্তু বাইরে:

```
{  
  let a = 10;  
  const b = 20;  
}  
  
console.log(a);  
console.log(b);
```

দুটোই inaccessible।

9. var vs let vs const Scope

Keyword	Scope
var	Function Scope
let	Block Scope
const	Block Scope

Example:

```
function test() {  
  var a = 10;  
  
  if (true) {  
    var b = 20;  
    let c = 30;  
    const d = 40;  
  }  
  
  console.log(a); // 10  
  console.log(b); // 20  
  console.log(c); // Error  
  console.log(d); // Error  
}
```

10. Nested Scope

একটি scope-এর ভিতরে আরেকটি scope থাকতে পারে।

```
const globalValue = "Global";

function outer() {
  const outerValue = "Outer";

  function inner() {
    const innerValue = "Inner";

    console.log(globalValue);
    console.log(outerValue);
    console.log(innerValue);
  }

  inner();
}

outer();
```

`inner()` function access করতে পারে:

- নিজের variable
- parent scope-এর variable
- global scope-এর variable

11. Lexical Scope

JavaScript-এর scope structure **code** কোথায় লেখা হয়েছে তার উপর নির্ভর করে।

এটাকে বলা হয় **Lexical Scope**।

```
const name = "Shariar";

function outer() {
  const age = 31;

  function inner() {
    console.log(name);
    console.log(age);
  }
}
```

```
}  
  
  inner();  
}
```

`inner()` function কোথায় লেখা হয়েছে সেটাই **determine** করে সে কোন **variables access** করতে পারবে।

12. Scope Chain

যদি JavaScript কোনো variable current scope-এ না পায়, তাহলে parent scope-এ খুঁজবে।

তারপর parent-এর parent scope-এ যাবে।

এই searching process-কে **Scope Chain** বলা হয়।

Example:

```
const a = "global";  
  
function outer() {  
  const b = "outer";  
  
  function inner() {  
    const c = "inner";  
  
    console.log(c);  
    console.log(b);  
    console.log(a);  
  }  
  
  inner();  
}
```

`inner()` যখন `a` খুঁজবে:

```
inner scope  
  ↓  
outer scope  
  ↓  
global scope
```

এটাই Scope Chain।

13. Scope Lookup Direction

Scope lookup সাধারণত:

Inner → Outer

হয়।

কিন্তু outer scope সরাসরি inner scope-এর variable দেখতে পারে না।

Example:

```
function outer() {  
  function inner() {  
    const secret = "123";  
  }  
  
  console.log(secret);  
}
```

এখানে error হবে।

কারণ `secret` inner scope-এ আছে।

Parent scope child-এর variable access করতে পারে না।

14. Variable Shadowing

যখন inner scope-এ একই নামের variable declare করা হয়, তখন inner variable outer variable-কে **shadow** করে।

```
const name = "Global";  
  
function test() {  
  const name = "Local";  
  
  console.log(name);  
}  
  
test();
```

Output:

Local

কারণ function-এর ভিতরের `name` outer `name` -কে shadow করছে।

Shadowing Example

```
let x = 10;

{
  let x = 20;

  console.log(x);
}

console.log(x);
```

Output:

20
10

দুটো `x` আলাদা variable।

15. Scope Quick Comparison

Scope	var	let	const
Global	✓	✓	✓
Function	✓	✓	✓
Block	✗	✓	✓

16. Hoisting কী?

Hoisting হলো JavaScript execution-এর আগে কিছু declarations-এর binding/environment তৈরি করে রাখার behavior।

সহজভাবে:

JavaScript code execute করার আগে declarations সম্পর্কে জানে।

কিন্তু গুরুত্বপূর্ণ বিষয়:

Hoisting মানে **code physically** উপরে উঠে যাওয়া নয়।

JavaScript-এর execution model বোঝার জন্য declaration এবং initialization আলাদা বিষয় হিসেবে ভাবতে হবে।

17. `var` Hoisting

`var` declaration hoisted হয় এবং সাধারণভাবে তার initial value থাকে:

```
undefined
```

Example:

```
console.log(x);  
  
var x = 10;
```

Output:

```
undefined
```

JavaScript-এর mental model:

```
var x;  
  
console.log(x);  
  
x = 10;
```

এখানে শুধু declaration আগে available হয়েছে; assignment পরে হয়েছে।

18. `var` Hoisting-এর গুরুত্বপূর্ণ বিষয়

```
console.log(x);  
  
var x = 100;  
  
console.log(x);
```

Output:

```
undefined  
100
```

কারণ:

```
var x;
```

আগেই environment-এ তৈরি হয়েছে।

কিন্তু:

```
x = 100;
```

পরে execute হয়েছে।

19. `let` Hoisting

অনেকে বলে:

```
"let hoisted হয় না।"
```

এটা পুরোপুরি accurate নয়।

`let` declaration-ও execution-এর আগে binding তৈরি করে, কিন্তু initialization-এর আগে access করা যায় না।

এই সময় variable **Temporal Dead Zone (TDZ)**-তে থাকে।

```
console.log(x);  
  
let x = 10;
```

Output:

ReferenceError

20. const Hoisting

`const` -এর ক্ষেত্রেও একই concept।

```
console.log(x);  
  
const x = 10;
```

Result:

ReferenceError

কারণ initialization-এর আগে access করা হচ্ছে।

21. var vs let vs const Hoisting

Keyword	Declaration Binding	Initialization	Before Declaration Access
var	Hoisted	undefined	undefined
let	Hoisted	declaration-এর জায়গায়	ReferenceError
const	Hoisted	declaration-এর জায়গায়	ReferenceError

এখানে "hoisted" বলতে execution-এর আগে binding তৈরি হওয়া বোঝানো হচ্ছে।

22. Function Declaration Hoisting

Function declaration সম্পূর্ণভাবে hoisted হওয়ায় declaration-এর আগে function call করা যায়।

```
greet();

function greet() {
  console.log("Hello");
}
```

Output:

```
Hello
```

এটা কাজ করে।

23. Function Declaration কেন আগে Call করা যায়?

JavaScript execution শুরু করার আগে function declaration-এর binding তৈরি করে এবং function body-এর সাথে সেটি associate করে।

তাই:

```
greet();
```

call করার সময় function available থাকে।

24. Function Expression এবং Hoisting

Function expression-এর ক্ষেত্রে behavior আলাদা।

```
greet();

const greet = function () {
  console.log("Hello");
};
```

এখানে:

ReferenceError

কারণ `greet` হলো `const` variable এবং initialization-এর আগে access করা হচ্ছে।

25. Arrow Function এবং Hoisting

Arrow function-ও সাধারণত variable-এর মাধ্যমে store করা হয়।

```
greet();  
  
const greet = () => {  
  console.log("Hello");  
};
```

এখানে:

ReferenceError

কারণ `const` TDZ-তে থাকে।

26. Function Declaration vs Function Expression

বিষয়	Function Declaration	Function Expression
Hoisting	Function available	Variable rules apply
Before declaration call	✓	সাধারণত ✗
Syntax	<code>function greet(){} </code>	<code>const greet = function(){} </code>
Arrow possible	✗	✓

27. Hoisting Mental Model

এই code:

```
console.log(a);
var a = 10;

sayHello();

function sayHello() {
  console.log("Hello");
}
```

Mental model:

```
var a;

function sayHello() {
  console.log("Hello");
}

console.log(a);
a = 10;

sayHello();
```

এটা **exact source transformation** নয়, শুধু behavior বোঝার mental model ।

28. Temporal Dead Zone — TDZ

TDZ = Temporal Dead Zone

`let` এবং `const` variable declaration-এর scope-এর শুরু থেকে declaration execute হওয়া পর্যন্ত সময়কে TDZ বলা হয় ।

Example:

```
console.log(name);

let name = "Sharir";
```

এখানে `name` scope-এ known, কিন্তু initialization হয়নি ।

তাই:

ReferenceError

29. TDZ কেন গুরুত্বপূর্ণ?

TDZ JavaScript-কে এমন error ধরতে সাহায্য করে যেখানে variable declaration-এর আগে accidentally ব্যবহার করা হয়েছে।

```
console.log(age);  
  
let age = 31;
```

JavaScript silently `undefined` না দিয়ে error দেয়।

এটা debugging-এর জন্য helpful।

30. TDZ-এর Timeline

এই code:

```
{  
  console.log(x);  
  
  let x = 10;  
}
```

Conceptually:

```
Block শুরু  
  ↓  
x binding exists  
  ↓  
TDZ  
  ↓  
let x = 10  
  ↓  
x initialized  
  ↓  
x ব্যবহার করা যাবে
```

31. Declaration vs Initialization

এই দুইটি আলাদা concept।

```
let age;
```

এখানে declaration হয়েছে এবং পরে initialization value হিসেবে `undefined` দেওয়া যায়:

```
age = 31;
```

অন্যদিকে:

```
let age = 31;
```

এখানে declaration + initialization একই statement-এ হয়েছে।

32. `let` TDZ Example

```
{  
  console.log(value);  
  let value = 50;  
}
```

Result:

```
ReferenceError
```

কারণ `value` declaration-এর আগে TDZ-তে।

33. `const` TDZ Example

```
{  
  console.log(value);  
}
```

```
const value = 50;
}
```

Result:

ReferenceError

34. TDZ শুধু Block-এর শুরু থেকে Declaration পর্যন্ত

```
{
  let x = 10;

  console.log(x);
}
```

এখানে TDZ নেই।

কারণ declaration execute হয়ে গেছে।

35. TDZ এবং typeof

অনেকে মনে করে:

```
typeof x
```

সবসময় safe।

কিন্তু TDZ-এর ক্ষেত্রে তা নয়।

```
console.log(typeof x);

let x = 10;
```

এখানে:

ReferenceError

কারণ `x` lexical binding-এর TDZ-তে আছে।

36. `var` এবং `typeof`

```
console.log(typeof x);  
  
var x = 10;
```

Output:

undefined

কারণ `var x` hoisted হয়ে `undefined` initialized state-এ থাকে।

37. TDZ Quick Comparison

```
console.log(a);  
var a = 10;
```

undefined

কিন্তু:

```
console.log(b);  
let b = 10;
```

ReferenceError

এবং:

```
console.log(c);  
const c = 10;
```

ReferenceError

38. Strict Mode কী?

JavaScript-এর **Strict Mode** হলো এমন একটি mode যেখানে JavaScript কিছু unsafe বা problematic behavior-এর ক্ষেত্রে আরও strict rules প্রয়োগ করে।

Enable করতে:

```
"use strict";
```

39. Strict Mode Basic Example

Normal mode-এ কিছু ভুল behavior silently কাজ করতে পারে।

Strict mode-এ একই কাজ error দিতে পারে।

Example:

```
"use strict";  
  
x = 10;
```

এখানে ☒ declare করা হয়নি।

Result:

ReferenceError

40. Strict Mode কেন ব্যবহার করা হয়?

Strict mode-এর লক্ষ্য:

- accidental mistakes কমানো
 - unsafe behavior detect করা
 - silent errors কমানো
 - কিছু legacy behavior restrict করা
 - code quality improve করা
-

41. Undeclared Variable

Strict mode-এ declaration ছাড়া variable assign করা যাবে না।

```
"use strict";  
  
username = "Shariar";
```

Result:

```
ReferenceError
```

সঠিক:

```
"use strict";  
  
let username = "Shariar";
```

42. Read-only Property Assignment

Strict mode কিছু assignment error স্পষ্টভাবে detect করে।

Example:

```
"use strict";  
  
const user = {};
```

```
Object.defineProperty(user, "name", {
  value: "Sharar",
  writable: false
});

user.name = "Duke";
```

এখানে strict mode-এ error হবে।

43. `this` এবং Strict Mode

Strict mode-এ regular function-এর `this` behavior পরিবর্তিত হয়।

```
"use strict";

function test() {
  console.log(this);
}

test();
```

এখানে `this` হবে:

```
undefined
```

Normal non-strict function call-এর ক্ষেত্রে browser environment-এ `this` global object-এর দিকে যেতে পারে।

44. Strict Mode এবং Functions

Strict mode পুরো script-এর জন্য:

```
"use strict";

function test() {
  // strict mode
}
```

অথবা function-এর ভিতরেও লেখা যায়:

```
function test() {  
  "use strict";  
  
  // strict mode  
}
```

45. Strict Mode এবং ES Modules

Modern JavaScript-এর **ES Modules** automatically strict mode-এ চলে।

Example:

```
// module.js  
  
export const name = "Sharir";
```

এখানে আলাদাভাবে:

```
"use strict";
```

লেখার প্রয়োজন নেই।

46. Scope + Hoisting + TDZ একসাথে

এখন সবচেয়ে গুরুত্বপূর্ণ example:

```
console.log(a);  
console.log(b);  
console.log(c);  
  
var a = 10;  
let b = 20;  
const c = 30;
```

Result:

```
undefined  
ReferenceError
```

b-তে error হওয়ার কারণে সাধারণ execution সেখানে থেমে যাবে।

Mental model:

```
var a  
  ↓  
initialized as undefined  
  
let b  
  ↓  
TDZ  
  
const c  
  ↓  
TDZ
```

47. গুরুত্বপূর্ণ Output Prediction

Example 1

```
console.log(a);  
  
var a = 10;
```

Answer:

```
undefined
```

Example 2

```
console.log(a);  
  
let a = 10;
```

Answer:

ReferenceError

Example 3

```
console.log(a);  
  
const a = 10;
```

Answer:

ReferenceError

Example 4

```
sayHello();  
  
function sayHello() {  
  console.log("Hello");  
}
```

Output:

Hello

Example 5

```
sayHello();  
  
const sayHello = () => {  
  console.log("Hello");  
};
```

Result:

ReferenceError

48. Scope + Shadowing Example

```
let name = "Global";

function outer() {
  let name = "Outer";

  function inner() {
    let name = "Inner";

    console.log(name);
  }

  inner();
}

outer();
```

Output:

Inner

কারণ JavaScript প্রথমে current scope-এ `name` খোঁজে।

49. Scope Chain Example

```
const a = 10;

function outer() {
  const b = 20;

  function inner() {
    const c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }
}
```

```
    }  
  
    inner();  
}  
  
outer();
```

`inner()` দেখতে পারে:

```
c → নিজের scope  
b → parent scope  
a → global scope
```

50. `var` Loop Problem-এর Foundation

`var` function-scoped হওয়ায় loops-এ কখনো unexpected behavior দেখা যায়।

```
for (var i = 0; i < 3; i++) {  
    console.log(i);  
}  
  
console.log(i);
```

Output:

```
0  
1  
2  
3
```

কারণ `var i` block-scoped নয়।

`let` হলে:

```
for (let i = 0; i < 3; i++) {  
    console.log(i);  
}  
  
console.log(i);
```

শেষের:

```
console.log(i);
```

error দেবে।

কারণ `let i` loop block-এর scope-এর মধ্যে।

51. কেন Modern JavaScript-এ `let` / `const` Prefer করা হয়?

পুরনো JavaScript-এ:

```
var
```

অনেক ব্যবহার করা হতো।

কিন্তু modern JavaScript-এ সাধারণভাবে:

```
const
```

default হিসেবে এবং প্রয়োজন হলে:

```
let
```

ব্যবহার করা হয়।

কারণ:

- block scope
 - TDZ
 - accidental redeclaration কম
 - code behavior predictable
 - loop behavior safer
-

52. `const` মানে **Immutable** নয়

এটা Scope chapter-এর গুরুত্বপূর্ণ related concept ।

```
const user = {  
  name: "Shariar"  
};  
  
user.name = "Duke";
```

এটা valid ।

কারণ `const` variable binding reassign করা যায় না:

```
user = {};
```

এটা error ।

কিন্তু object-এর property পরিবর্তন করা যায় ।

53. Scope-এর Mental Model

একটি বড় building চিন্তা করো ।

```
Global Scope  
|  
├── Function Scope  
|   |  
|   ├── Block Scope  
|   └── Block Scope  
└── Another Scope
```

Inner scope সাধারণত outer scope-এর variables access করতে পারে ।

কিন্তু outer scope inner scope-এর variables দেখতে পারে না ।

54. Hoisting-এর Mental Model

Hoisting-কে এভাবে মনে রাখো:

```
JavaScript শুরু
↓
Declarations সম্পর্কে environment তৈরি
↓
Variables/functions-এর binding প্রস্তুত
↓
Code execution শুরু
```

কিন্তু variable-এর ক্ষেত্রে declaration এবং initialization একই জিনিস নয়।

55. TDZ-এর Mental Model

```
let / const declaration-এর scope শুরু
↓
TDZ
↓
declaration execute
↓
initialization
↓
variable usable
```

56. Scope বনাম Hoisting বনাম TDZ

Concept	মূল প্রশ্ন
Scope	Variable কোথায় accessible?
Hoisting	Execution-এর আগে declaration সম্পর্কে কীভাবে environment তৈরি হয়?
TDZ	<code>let</code> / <code>const</code> initialization-এর আগে কেন access করা যায় না?
Strict Mode	JavaScript-এর কিছু problematic behavior কীভাবে stricter করা যায়?

57. Common Mistakes

Mistake 1 — Hoisting মানে code উপরে চলে যায়

❌ ভুল ধারণা:

JavaScript code physically উপরে move করে।

✅ সঠিক:

JavaScript execution-এর আগে declarations-এর জন্য bindings/environment তৈরি করে।

Mistake 2 — `let` এবং `const` hoisted হয় না

এভাবে বলা oversimplification।

Better understanding:

`let` এবং `const` binding তৈরি হয়, কিন্তু initialization-এর আগে TDZ-তে থাকে।

Mistake 3 — `var` block scoped

```
if (true) {  
  var x = 10;  
}  
  
console.log(x);
```

এটা কাজ করবে।

কারণ `var` function-scoped।

Mistake 4 — `const` object পরিবর্তন করা যায় না

```
const user = {  
  name: "A"  
};
```

```
user.name = "B";
```

এটা valid।

`const` binding reassign করতে দেয় না।

Mistake 5 — `typeof` সবসময় safe

TDZ-এর ক্ষেত্রে:

```
console.log(typeof x);  
  
let x = 10;
```

`ReferenceError` হবে।

58. Best Practices

1. Default হিসেবে `const` ব্যবহার করো

```
const name = "Shariar";
```

2. Value reassign করতে হলে `let`

```
let count = 0;  
  
count++;
```

3. নতুন code-এ সাধারণত `var` avoid করো

```
const  
let
```

Prefer করো।

4. Variable declaration-এর আগে **access** করা **avoid** করো

যদিও কিছু ক্ষেত্রে `var` technically কাজ করে:

```
console.log(value);  
  
var value = 10;
```

এটা readability-এর জন্য ভালো pattern নয়।

5. Scope যত ছোট রাখা যায় তত ভালো

```
function calculate() {  
  const total = 100;  
  
  return total;  
}
```

অপ্রয়োজনীয় global variable তৈরি করো না।

59. Real-World Example — User Data

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
function getUsername() {  
  return user.name;  
}  
  
console.log(getUsername());
```

এখানে:

```
Global Scope  
  ↓  
user  
  
Function Scope
```

↓
getUser_name()

Function global `user` access করতে পারছে।

60. Real-World Example — Local Variable

```
function calculateTotal(price, quantity) {  
  const total = price * quantity;  
  
  return total;  
}  
  
console.log(calculateTotal(500, 2));
```

`total` function-এর বাইরে দরকার নেই।

তাই local scope-এ রাখা ভালো।

61. Real-World Example — Block Scope

```
function login(user) {  
  if (user) {  
    const message = "User logged in";  
  
    console.log(message);  
  }  
}
```

`message` শুধু `if` block-এ দরকার।

তাই block scope-এ রাখা clean design।

62. Interview Question — `var`, `let`, `const`

Question:

What is the difference between `var`, `let`, and `const`?

Answer:

- `var` is function-scoped.
 - `let` and `const` are block-scoped.
 - `var` can be redeclared.
 - `let` cannot be redeclared in the same scope.
 - `const` cannot be reassigned.
 - `let` and `const` have TDZ behavior.
 - `var` is initialized as `undefined` when its declaration is processed.
-

63. Interview Question — Hoisting

Question:

What is hoisting?

Answer:

Hoisting refers to JavaScript's behavior where declarations are processed before the execution of the surrounding code, making certain bindings available according to their declaration type.

64. Interview Question — TDZ

Question:

What is Temporal Dead Zone?

Answer:

TDZ is the period between entering the scope of a `let` or `const` declaration and the point where that declaration is initialized. Accessing the variable during this period throws a `ReferenceError`.

65. Interview Question — Why Does `var` Return Undefined?

```
console.log(x);
```

```
var x = 10;
```

Answer:

`var x` is initialized with `undefined` before execution reaches the assignment.

Conceptual model:

```
var x;  
  
console.log(x);  
  
x = 10;
```

66. Interview Question — Function Declaration Hoisting

```
greet();  
  
function greet() {  
  console.log("Hello");  
}
```

Why does this work?

Because function declarations are available during the relevant initialization phase before normal statement execution.

67. Practice — Output Prediction

Question 1

```
console.log(a);  
  
var a = 10;
```

What happens?

Question 2

```
console.log(a);  
  
let a = 10;
```

What happens?

Question 3

```
console.log(a);  
  
const a = 10;
```

What happens?

Question 4

```
sayHi();  
  
function sayHi() {  
  console.log("Hi");  
}
```

What happens?

Question 5

```
{  
  let x = 10;  
}  
  
console.log(x);
```

What happens?

Question 6

```
{  
  var x = 10;  
}  
  
console.log(x);
```

What happens?

68. Practice — Scope Challenge

Predict the output:

```
let x = "A";  
  
function test() {  
  let x = "B";  
  
  if (true) {  
    let x = "C";  
    console.log(x);  
  }  
  
  console.log(x);  
}  
  
test();  
  
console.log(x);
```

Think about:

```
Block Scope  
  ↓  
Function Scope  
  ↓  
Global Scope
```

69. Coding Challenge — Scope

Create a function:

```
function calculateDiscount(price) {  
  // your code  
}
```

Requirements:

- `price` function parameter হবে
- function-এর ভিতরে discount rate declare করবে
- final price return করবে
- unnecessary global variable ব্যবহার করবে না

Example:

```
calculateDiscount(1000);
```

Expected:

```
900
```

70. Coding Challenge — Hoisting Prediction

নিজে answer করার চেষ্টা করো:

```
console.log(a);  
console.log(b);  
  
var a = 10;  
let b = 20;
```

Question:

1. প্রথম `console.log()` কী output দেবে?
 2. দ্বিতীয়টি কি execute হবে?
 3. কেন?
-

71. Coding Challenge — Scope Chain

এই code modify করো:

```
const appName = "Shop";

function app() {
  const version = "1.0";

  function showInfo() {
    // এখানে appName এবং version access করো
  }

  showInfo();
}

app();
```

Goal:

`showInfo()` থেকে outer এবং global variable access করা।

72. Quick Revision

Scope

Global
Function
Block
Lexical
Scope Chain

Variable Scope

var → Function Scope
let → Block Scope
const → Block Scope

Hoisting

```
var  
→ binding processed  
→ initialized as undefined
```

```
let / const  
→ binding processed  
→ TDZ  
→ initialization
```

Function declaration:

```
function declaration  
→ available before normal statement execution
```

TDZ

```
let / const  
  ↓  
scope starts  
  ↓  
TDZ  
  ↓  
declaration / initialization  
  ↓  
usable
```

Strict Mode

```
"use strict";
```

মূল উদ্দেশ্য:

- accidental errors detect করা
- undeclared assignment prevent করা
- কিছু problematic legacy behavior restrict করা
- safer JavaScript code লেখা

73. Golden Rules 🧠

Rule 1: `var` = function scoped.

Rule 2: `let` এবং `const` = block scoped.

Rule 3: Scope determines **where** a variable can be accessed.

Rule 4: Inner scope outer scope-এর variable access করতে পারে।

Rule 5: Outer scope inner scope-এর local variable access করতে পারে না।

Rule 6: Hoisting মানে source code physically উপরে উঠে যাওয়া নয়।

Rule 7: `var` declaration processed হলে সাধারণভাবে `undefined` initialization থাকে।

Rule 8: `let` এবং `const` initialization-এর আগে TDZ-তে থাকে।

Rule 9: TDZ-তে access করলে `ReferenceError` হয়।

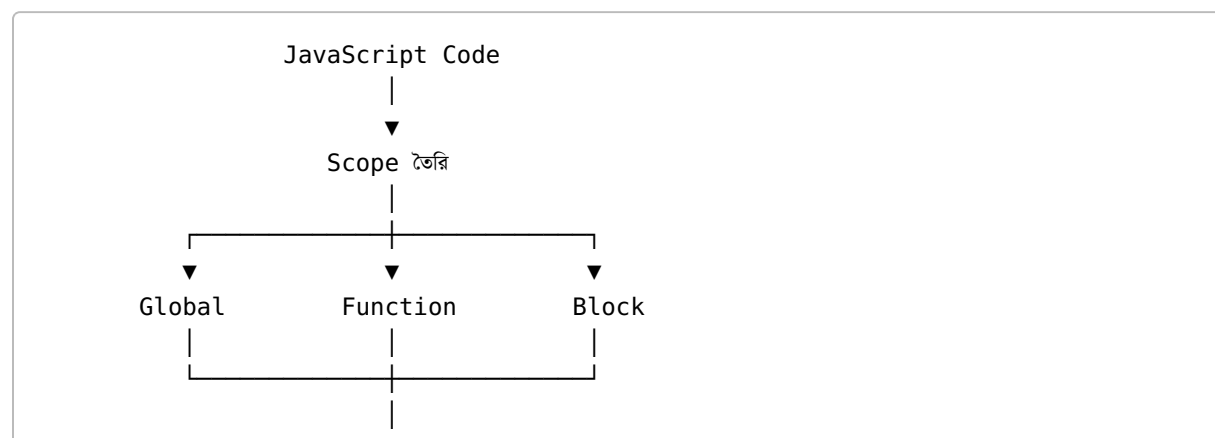
Rule 10: Function declarations normal statement execution-এর আগে available হতে পারে।

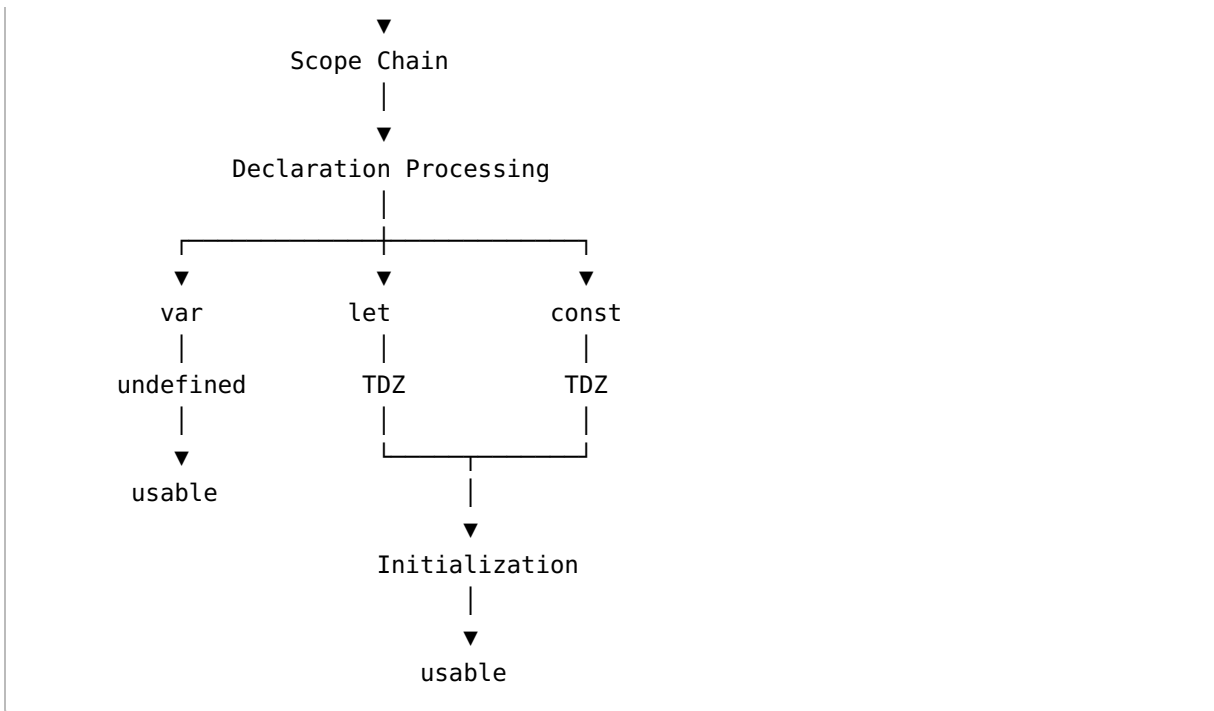
Rule 11: Modern JavaScript-এ সাধারণত `const` এবং `let` prefer করে।

Rule 12: `"use strict"` JavaScript-কে আরও strict behavior-এ চালাতে সাহায্য করে।

74. Final Mental Model

পুরো Section-টা একসাথে এভাবে মনে রাখো:





আর Strict Mode:

```

"use strict";
  ↓
Stricter JavaScript Rules
  ↓
Fewer silent mistakes
  ↓
More predictable code
  
```

75. Section 11 Complete Checklist

তুমি এই section শেষ করার পর এগুলো explain করতে পারলে তোমার foundation strong:

- ☐ Global Scope
- ☐ Function Scope
- ☐ Block Scope
- ☐ Lexical Scope
- ☐ Scope Chain
- ☐ Nested Scope
- ☐ Variable Shadowing
- ☐ `var` scope
- ☐ `let` scope

- [] `const` scope
- [] Hoisting
- [] `var` hoisting
- [] `let` / `const` hoisting behavior
- [] Function declaration hoisting
- [] Function expression hoisting behavior
- [] Arrow function hoisting behavior
- [] TDZ
- [] Declaration vs Initialization
- [] `typeof` + TDZ
- [] Strict Mode
- [] `"use strict"`
- [] Undeclared variable behavior
- [] Strict mode + `this` basics
- [] ES Modules and strict mode
- [] Scope vs Hoisting vs TDZ difference

Section 11 Summary

সবচেয়ে important চারটি connection:

Scope

↓

কোথায় variable accessible?

Hoisting

↓

Execution-এর আগে declaration-এর binding কীভাবে প্রস্তুত হয়?

TDZ

↓

`let/const` initialization-এর আগে কেন access করা যায় না?

Strict Mode

↓

কিছু problematic JavaScript behavior কেন আরও strictভাবে ধরা হয়?

এই চারটি concept ভালোভাবে বুঝে গেলে পরের advanced JavaScript topics—বিশেষ করে **Closures**, **Execution Context**, `this`, এবং **asynchronous JavaScript**—বোঝা অনেক সহজ হয়ে যাবে।